day 23 | 251022

Today we will continue to work with Euler's Method for solving differential equations, but this time we will turn to python to solve this problem now that we have some experience with Excel. We will talk about how to handle these first order differential equations that have only one variable on the right hand side, and then we will talk about how to handle them if they have two variables. These look like these two equations:

$$\frac{dy}{dx} = x^2 + x - 1 \tag{1}$$

$$\frac{dy}{dx} = y^3 + \sin x \tag{2}$$

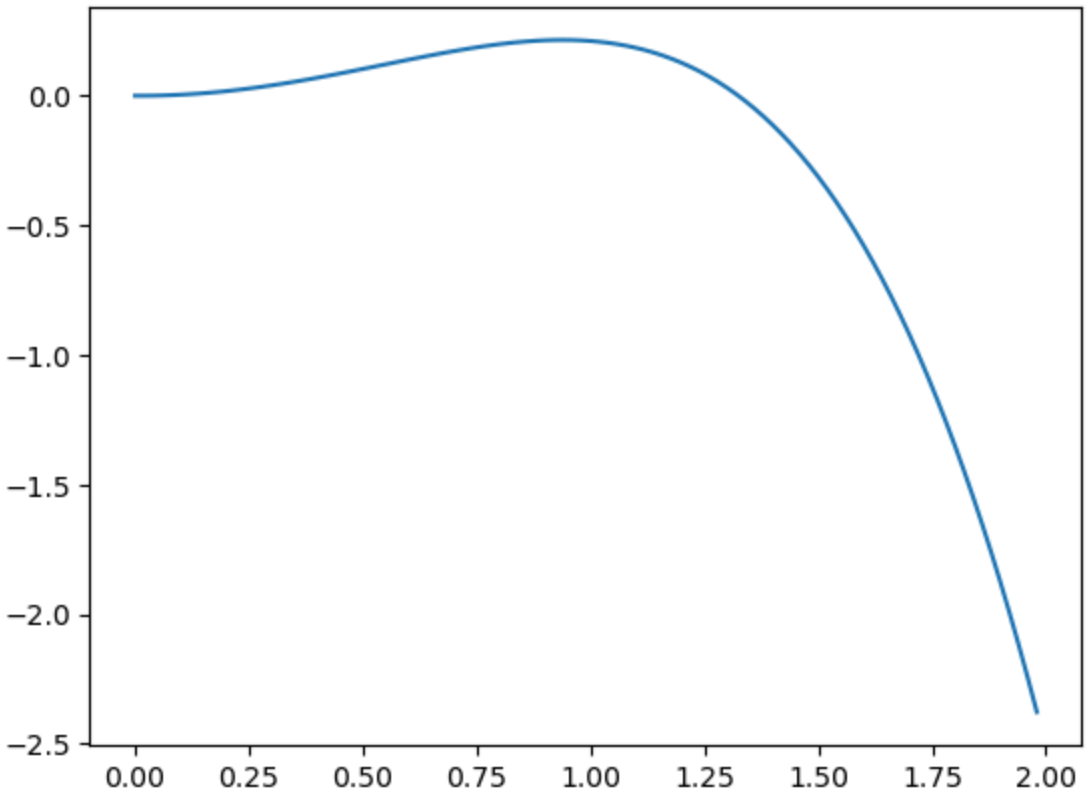
Euler's Method is good, but the error grows over time as we have seen. This is part of every solution to differential equations, but it is something that we want to minimize. Next time, we will discuss some methods that will be better than Euler's method and won't take much longer to work through.

In [32]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [33]:
```
# define the differential equation
def f(x_): # dy/dx
    return(-x_**3+np.sin(x_))

# define the boundary condition
a = 0.0
b = 2.0
N = 100
dx = (b-a)/N

y = 0.0 # initial condition

xpoints = np.arange(a, b, dx)
ypoints = []

for i in xpoints:
    ypoints.append(y)
    y = y + f(i) * dx
```

In [34]:
```
fig0, ax0 = plt.subplots()
ax0.plot(xpoints, ypoints)
```

Out[34]:
```
[<matplotlib.lines.Line2D at 0x7ac530d40cb0>]
```



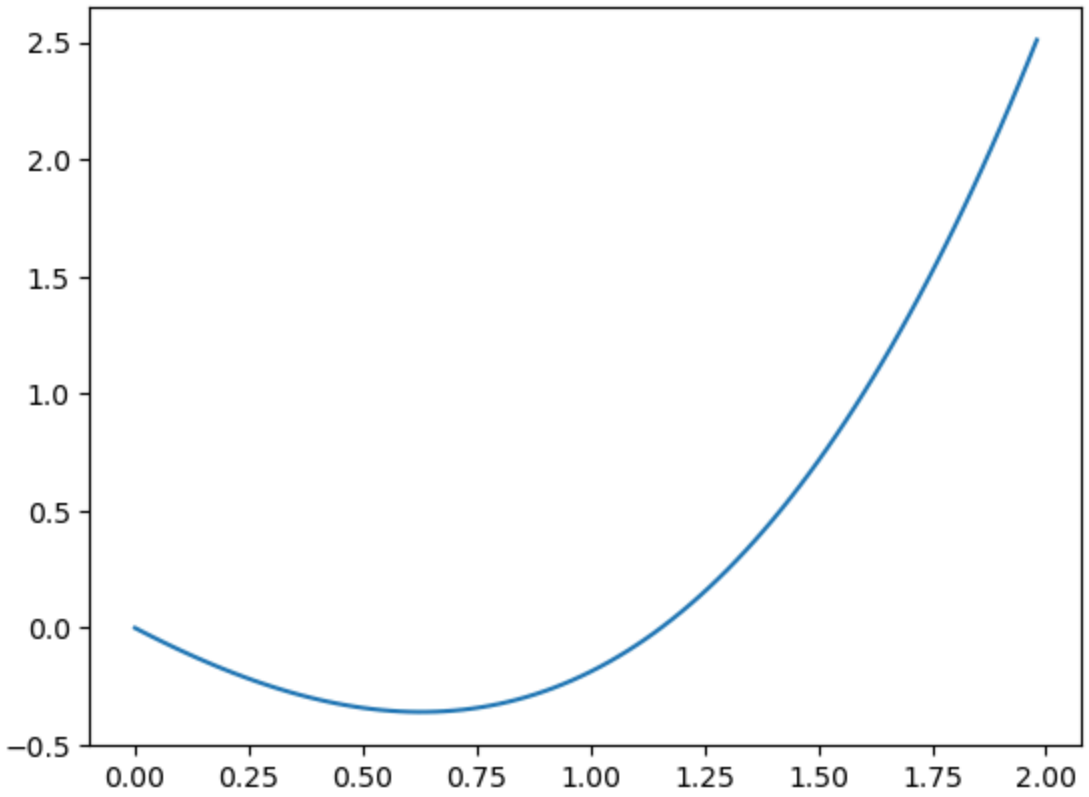
Or how about an equation that we have already solved with excel! Like this one:

$$\frac{dy}{dx} = x^2 + x - 1$$

In [35]:
```
# define the differential equation
def f(x): # dy/dx
    return(x**2+x-1)

# define the boundary condition
a = 0.0
b = 2.0
N = 100
dx = (b-a)/N

y = 0.0 # initial condition

xpoints = np.arange(a, b, dx)
ypoints = []

for i in xpoints:
    ypoints.append(y)
    y = y + f(i) * dx
```

In [36]:
```
fig1, ax1 = plt.subplots()
ax1.plot(xpoints, ypoints)
```

Out[36]:
```
[<matplotlib.lines.Line2D at 0x7ac530d93cb0>]
```

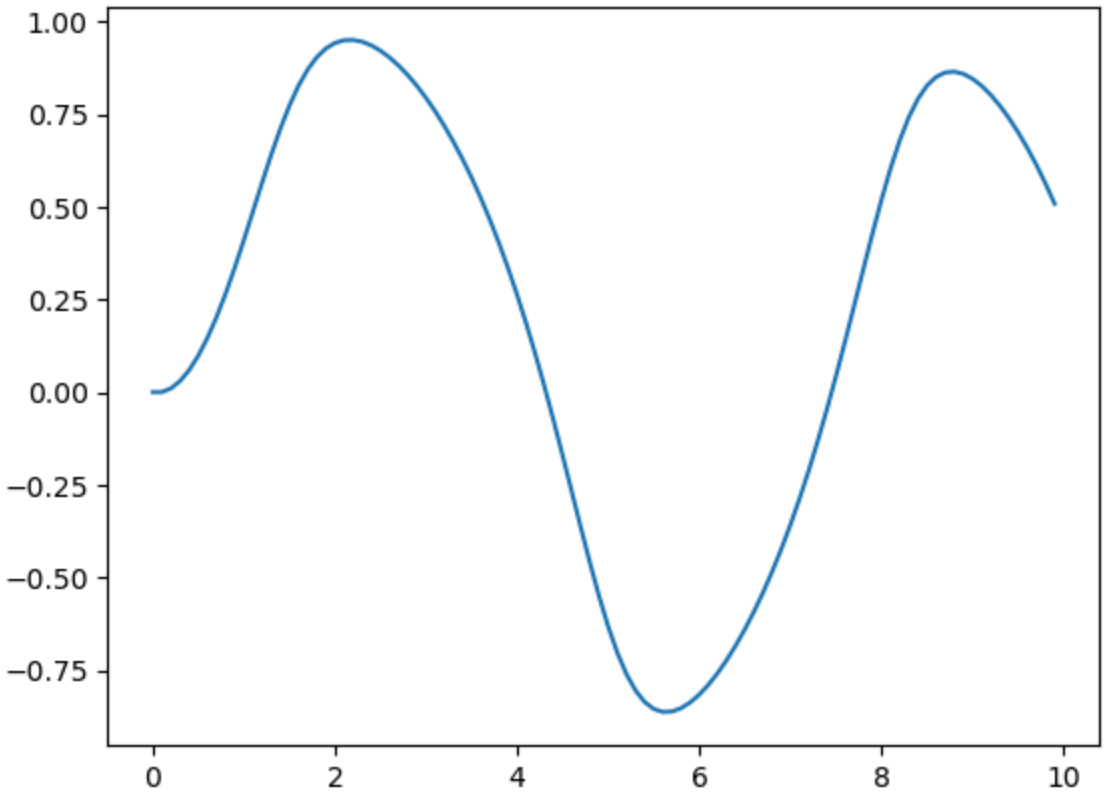


Now what if we change our equation (seemingly only slightly!), and we look at what we get now. The differential equation is now a function of two variables! But that is ok we can handle that!

In [37]:
```
# define the differential equation
def f(x_, y_): # dy/dx
    return(-y_**3+np.sin(x_))

# define the boundary condition
a = 0.0
b = 10.0
N = 100
dx = (b-a)/N

y = 0.0 # initial condition

xpoints = np.arange(a, b, dx)
ypoints = []

for i in xpoints:
    ypoints.append(y)
    y = y + f(i, y) * dx
```

In [38]:
```
fig2, ax2 = plt.subplots()
ax2.plot(xpoints, ypoints)
```

Out[38]:
```
[<matplotlib.lines.Line2D at 0x7ac530c3c2f0>]
```



In [ ]: